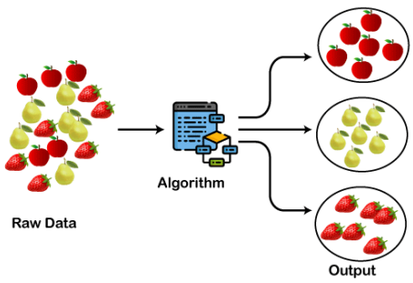
Part 1: Real-World Applications of K-Means



Task 1: Real-World Scenario – Fruit Classification

K-Means clustering can be used in agriculture or food processing industries to classify fruits based on physical features like color, size, shape, or texture. For example, apples and oranges can be grouped by measuring features such as diameter, weight, and color values (RGB or HSV). K-Means works by taking this numerical data and assigning each fruit to a cluster based on similarity. The algorithm calculates the centroid of each cluster and adjusts the groups iteratively to minimize the distance between data points and their cluster center. This helps in +-- automatically sorting or labeling fruits without needing manual inspection or predefined categories.

Task 2: Benefits of Using K-Means

One benefit is **automation of sorting** in fruit factories, which speeds up the process and reduces manual labor. Another benefit is that it can **handle large volumes of fruit data** easily, simplifying the classification process without needing labeled examples like in supervised learning.

K-Means vs K-NN: Key Differences

Feature	K-Means Clustering	K-NN (K-Nearest Neighbours)
Type	Unsupervised Learning	Supervised Learning
Purpose	To group similar data into clusters	To classify or predict a label for new data
Input Data	Only features (no labels)	Features and labels are needed for training
Output	Cluster assignments (e.g., Cluster 1, 2, 3)	Predicted class (e.g., Apple or Banana)
How It Works	Finds k centroids, assigns points to the nearest centroid	Finds the k closest labeled points and does majority vote
Training	Learns from data structure	Memorizes training data (lazy learning)
Example Use	Fruit grouping based on weight & color	Predict fruit type (Apple, Banana) from features

K-Means Simulation: Fruit Classification Step 1: Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import seaborn as sns
import os

sns.set()
os.environ["OMP_NUM_THREADS"] = "1"
```

Step 2: Create Sample Data

```
# Simulated fruit data (weight in grams, size in cm, color intensity as number)
# 3 types of fruits: Apples, Oranges, Bananas
fruit_data = {
    'Weight': [150, 160, 170, 180, 120, 130, 140, 145, 110, 115, 125, 105],
    'Size': [7.0, 7.5, 8.0, 8.5, 6.0, 6.2, 6.5, 6.8, 9.0, 9.2, 9.5, 9.8],
    'Color_Intensity': [200, 210, 190, 195, 220, 225, 230, 215, 160, 165, 155, 150]
}

df = pd.DataFrame(fruit_data)
df.head()
```

	Weight	Size	Color_Intensity
0	150	7.0	200
1	160	7.5	210
2	170	8.0	190
3	180	8.5	195
4	120	6.0	220

Step 3: Scale the Features

```
scaler = StandardScaler()
scaled_features = scaler.fit_transform(df)
```

Step 4: Apply K-Means Clustering

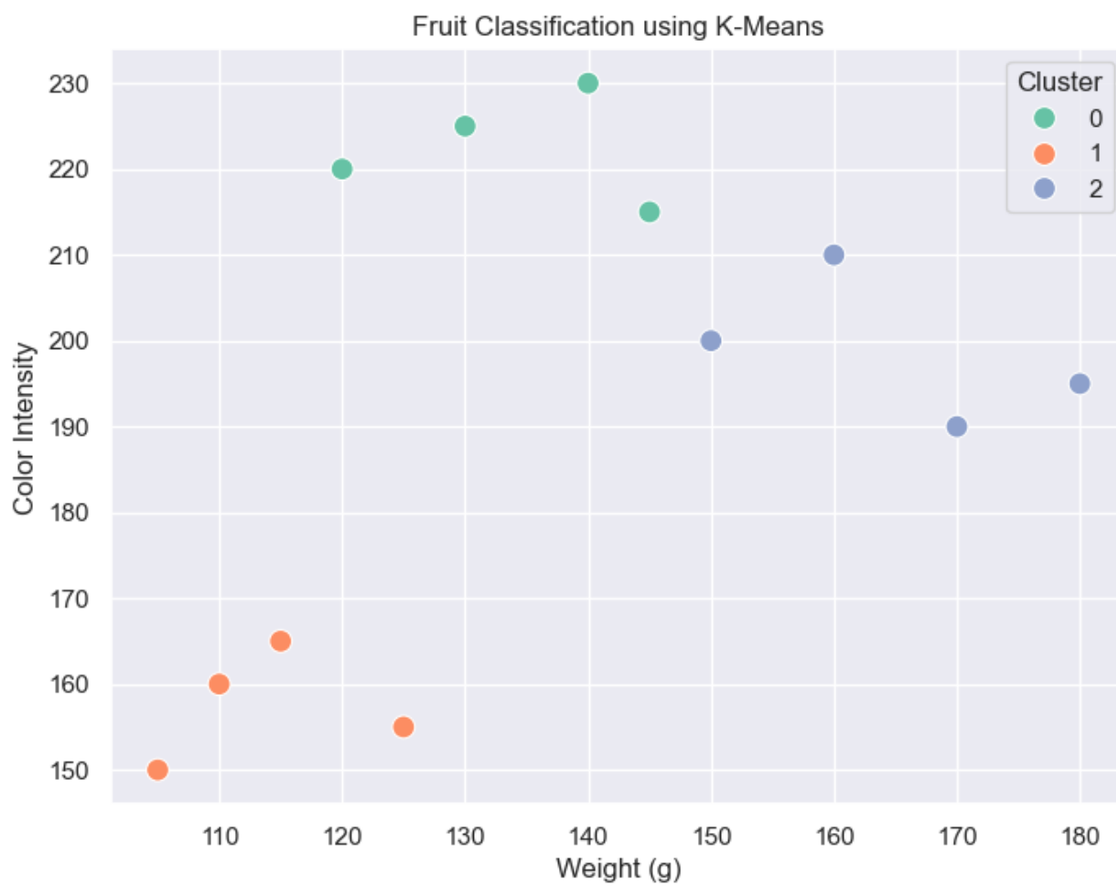
```
kmeans = KMeans(n_clusters=3, random_state=42)
clusters = kmeans.fit_predict(scaled_features)

df['Cluster'] = clusters
```

```
C:\Users\yerra\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows
with MKL, when there are less chunks than available threads. You can
avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```

Step 5: Visualize the Clusters

```
# 2D scatter plot using Weight vs Color Intensity
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='Weight', y='Color_Intensity', hue='Cluster', palette='Set2', s=100)
plt.title('Fruit Classification using K-Means')
plt.xlabel('Weight (g)')
plt.ylabel('Color Intensity')
plt.grid(True)
plt.show()
```



Optional: Check Cluster Centers

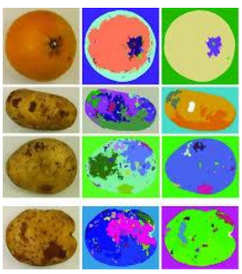
```
print("Cluster Centers (after scaling):")
print(kmeans.cluster_centers_)
```

```
Cluster Centers (after scaling):
[[-0.16206354 -1.12886781  1.07292873]
 [-1.02640243  1.19337455 -1.28449215]
 [ 1.18846597 -0.06450673  0.21156341]]
```

1. Customer Segmentation → Fruit Buyer Profiling

In fruit classification, this can be applied **after classification**, to group **buyers based on their fruit preferences**:

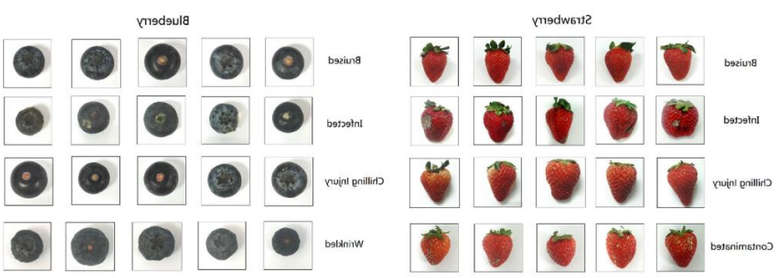
- Example: After classifying apples, oranges, and bananas, K-Means can be used to segment customers who prefer certain fruits.
- Use: Helps design **targeted offers**, like discounts on bananas for banana-loving customers.



2. Image Compression → Fruit Image Simplification

In fruit classification using **images**, K-Means can be used to **compress or simplify fruit images** by reducing color complexity:

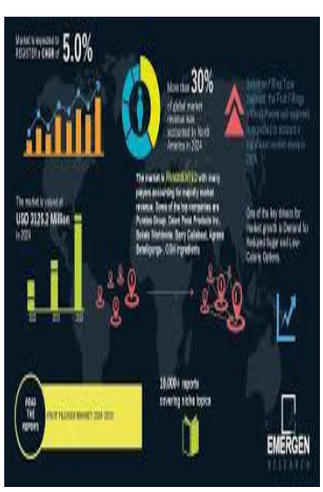
- Example: A fruit image with 1,000 colors can be compressed to 5 main colors using K-Means.
- Use: Makes image-based fruit classification models faster and lighter.



3. Anomaly Detection → Detecting Rotten or Misplaced Fruits

K-Means can help spot fruits that don't belong to any cluster well (outliers):

- Example: A fruit with unusual weight and color may be **rotten** or **misclassified**.
- Use: Helps in **quality control** and **removing spoiled fruits** from packaging lines.



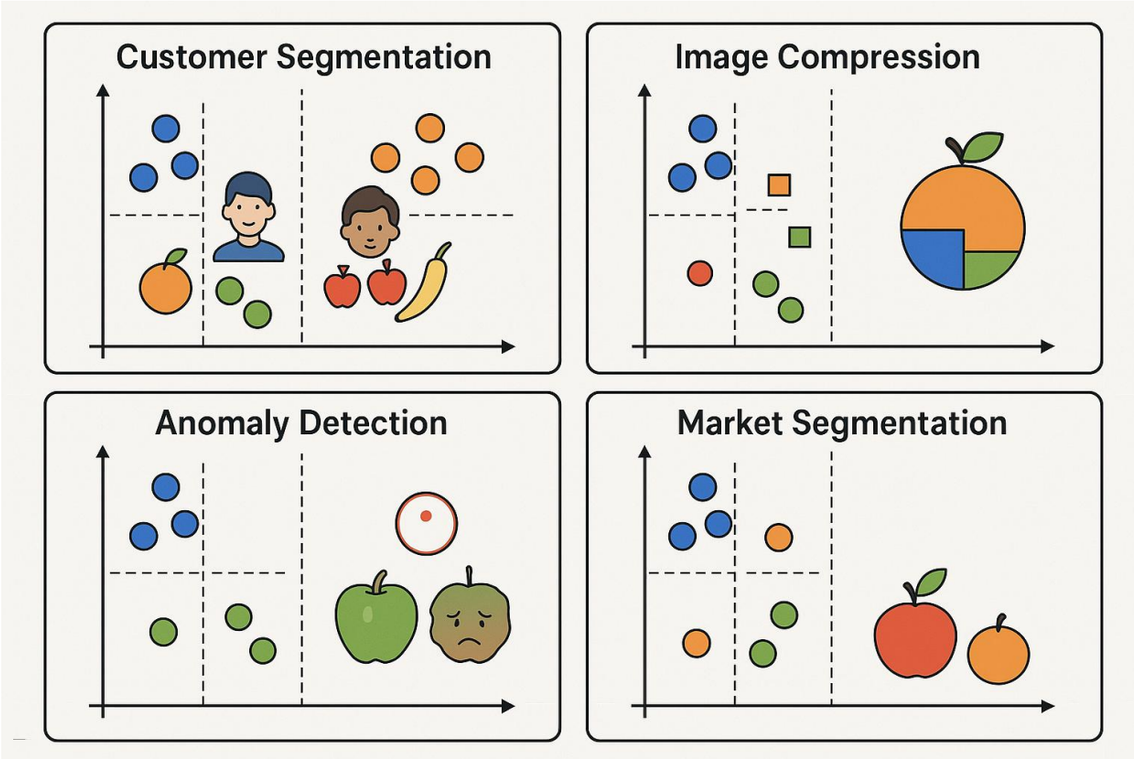
4. Market Segmentation → Grouping Fruit Types for Sales

Once fruits are classified, K-Means can group them based on market data (like demand, price range, and size):

- Example: Small apples sold in budget markets vs. large premium apples.
- Use: Helps retailers **optimize pricing** and **distribution**.

In summary:

K-Means isn't just useful for raw fruit classification. It can **enhance post-classification applications** in marketing, image processing, quality control, and customer analytics.



Part 2: Challenges and Alternatives

Task 1: Limitations of K-Means Clustering

K-Means assumes that all clusters are **spherical and evenly sized**, which becomes a limitation when classifying fruits that vary significantly in shape, size, or density. For example, bananas and grapes may not form round, compact clusters due to their unique features. Another key limitation is its **sensitivity to the initial placement of centroids**. If the centroids are not initialized properly, K-Means may converge to a suboptimal solution and produce different results on different runs.

Task 2: When Not to Use K-Means

K-Means is not suitable when the dataset contains **irregularly shaped clusters or overlapping classes**, such as differentiating between similar-looking fruits like tangerines and oranges. In these situations, more flexible algorithms like **Hierarchical**

Feature	Hierarchical Clustering	DBSCAN
Clustering Approach	Builds a hierarchy of clusters	Groups based on density
Number of Clusters	Determined by dendrogram, not pre-specified	Determined automatically based on density
Noise/Outliers	Sensitive, every point is clustered	Robust, identifies and marks as noise
Cluster Shape	May struggle with non-spherical clusters	Robust to irregular shapes
Computational Cost	Can be high, especially for large datasets	Can be efficient for large datasets
Parameter Tuning	Less sensitive to parameters	Sensitive to epsilon and minPts

Clustering or **DBSCAN** are preferred. These methods can detect clusters of varying shapes and densities and do not require specifying the number of clusters beforehand, making